Runtime and network

8B. Runtime & Network

Questions: 7 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft

---

Q 8.6 How do you optimize JavaScript execution? Explain long tasks, `scheduler.yield()`, and `requestIdleCallback`.

Difficulty: Senior **Asked at:** Google, Amazon, Microsoft

The browser's main thread handles JavaScript execution, DOM updates, event handling, layout, and painting — all on one thread. A JavaScript function that takes 200ms to run blocks everything else for 200ms. No scrolling, no clicking, no animations. Users perceive anything over 50ms as laggy.

Long tasks: Any task that takes >50ms on the main thread. Chrome DevTools flags them with red triangles in the Performance panel.

Breaking up long tasks with `scheduler.yield()`:

JAVASCRIPT Copy

```
// BAD: One long task blocks the main thread for 500ms
function processAllItems(items) {
  items.forEach(item => heavyProcessing(item)); // 500ms solid block
}
```

```
  for (let i = 0; i < items.length; i++) {
    heavyProcessing(items[i]);

    if (i % 100 === 0) {
      await scheduler.yield();
      // Browser can handle clicks, scrolls, and paint between chunks
    }
  }
}
```

`scheduler.yield()` pauses your function, lets the browser handle user input and rendering, then resumes. This keeps INP low because the browser can respond to interactions between chunks. It was introduced in 2024 and is the modern replacement for the `setTimeout(0)` hack.

`requestIdleCallback` **— low-priority work:**

JAVASCRIPT Copy

```
// Run non-critical work when the browser has free time
requestIdleCallback((deadline) => {
  while (deadline.timeRemaining() > 0 && analyticsQueue.length > 0) {
    sendAnalyticsEvent(analyticsQueue.shift());
  }

  if (analyticsQueue.length > 0) {
    requestIdleCallback(processQueue); // Continue in next idle period
  }
});
```

Use for: analytics, prefetching, non-visible DOM updates, background data processing. Never use for anything the user is waiting for — there's no guarantee when idle time will be available.

Web Workers for heavy computation:

When `scheduler.yield()` isn't enough (the computation itself takes seconds), move it entirely off the main thread:

```
// Sorting 100,000 items — always off the main thread
const worker = new Worker(new URL('./sort-worker.js', import.meta.url));
worker.postMessage(largeArray);
worker.onmessage = (e) => renderResults(e.data);
```

Decision tree for JavaScript execution optimization:

- Task takes <50ms → Leave it as is
- Task takes 50–200ms → Break it up with `scheduler.yield()` or `setTimeout(0)`
- Task takes 200ms+ → Move to a Web Worker
- Task is non-critical → Use `requestIdleCallback`

Q 8.7 What CSS properties affect performance? Explain containment, `content-visibility`, and `will-change`.

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart

Covered partially in Chapter 3 (Q3.10). Here we focus on the CSS properties specifically designed for performance.

CSS Containment (`contain`):

Tells the browser that a subtree of the DOM is independent — changes inside it don't affect anything outside. The browser can optimize by skipping work on unaffected areas.

css Copy

```
.card {
  contain: layout style;
  /* layout: This element's layout doesn't affect siblings */
  /* style: Counters and quotes don't escape this element */
}

/* Most aggressive: contain everything */
```

```
  /* The element's size must be set explicitly (won't auto-size to content
}
```

`content-visibility: auto` — **the biggest single performance win for long pages:**

```css
.section {
  content-visibility: auto;
  contain-intrinsic-size: 0 500px;
}
```

This tells the browser: "Don't render this section's content until it's near the viewport." The browser skips layout, paint, and style calculation for offscreen sections entirely. The `contain-intrinsic-size` provides an estimated height so the scrollbar is still accurate.

On a page with 50 sections, this can reduce initial rendering work by 50-90%. The browser only renders the 2-3 visible sections and defers the rest until the user scrolls.

`will-change` — **GPU layer promotion:**

```css
/* Tell the browser this element will animate — create a GPU layer ahead o
.card {
  will-change: transform;
}

.card:hover {
  transform: scale(1.05); /* Animation starts instantly — layer was pre-cr
}
```

Use sparingly — each GPU layer consumes memory. Don't apply `will-change` to every element. Apply it to elements you know will animate, and ideally add it dynamically just before the animation and remove it after.

Q 8.8 How does browser caching work? Explain `Cache-Control`, `ETag`, and `stale-while-revalidate`.

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Razorpay

Browser caching avoids re-downloading resources that haven't changed. The server controls caching through HTTP headers.

`Cache-Control` — **the primary caching header:**

JAVASCRIPT Copy

```
# Cache for 1 year (immutable assets with hashed filenames)
Cache-Control: public, max-age=31536000, immutable

# Cache for 5 minutes, revalidate after
Cache-Control: public, max-age=300

# Don't cache at all
Cache-Control: no-store

# Cache but always revalidate before using
Cache-Control: no-cache
```

`no-cache` doesn't mean "don't cache." It means "cache it, but check with the server before using it." The server returns 304 Not Modified if unchanged (saving bandwidth) or the new content if changed.

`no-store` means genuinely don't cache — for sensitive data (banking, health records).

`ETag` — **fingerprint-based revalidation:**

JAVASCRIPT Copy

```
# Server sends ETag with the response
ETag: "abc123"
Cache-Control: no-cache

# Browser's next request includes the ETag
If-None-Match: "abc123"
```

```
# No → 304 Not Modified (no body, save bandwidth)
# Yes → 200 OK with new content and new ETag
```

`stale-while-revalidate` — **serve stale, refresh in background:**

JAVASCRIPT Copy

```
Cache-Control: public, max-age=300, stale-while-revalidate=600
```

For the first 300 seconds, the cached response is "fresh" — served without contacting the server. From 300-900 seconds, the cached response is "stale but usable" — served immediately while the browser fetches a fresh version in the background. After 900 seconds, the cache entry is too old and the browser must wait for a fresh response.

Practical caching strategy for a frontend app:

JAVASCRIPT Copy

```
Resource                | Cache-Control
------------------------+-----------------------------------------
HTML pages              | no-cache (always revalidate - content changes)
JS/CSS with hash        | public, max-age=31536000, immutable
(app.a1b2c3.js)         | (hash changes when content changes)
Images with hash        | public, max-age=31536000, immutable
API responses           | private, max-age=60, stale-while-revalidate=30
User-specific data      | private, no-cache (or no-store for sensitive d
```

The pattern: HTML is never cached aggressively (it's the entry point that references versioned assets). JS/CSS/images use content hashing — `app.a1b2c3.js` has a unique filename per version, so it can be cached forever. When you deploy new code, the HTML references a new hash (`app.d4e5f6.js`), and the browser downloads the new file.

Follow-up: "What's the difference between `public` and `private`?"

`public` allows CDNs and proxies to cache the response (shared cache). `private` means only the user's browser can cache it — not CDNs. Use `private` for user-specific

Q 8.9 What network optimizations should every frontend engineer know?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Swiggy

HTTP/2 multiplexing — covered in Chapter 5 (Q5.4). Single connection, multiple streams, no domain sharding needed.

Resource hints — covered in Chapter 1 (Q1.10). `preload` for critical resources, `prefetch` for next-page resources, `preconnect` for third-party origins.

`fetchpriority` **— controlling download priority:**

HTML Copy

```
<!— High priority: LCP image should download before other images —>
<img src="/hero.jpg" fetchpriority="high" alt="Hero" />

<!— Low priority: footer image can wait —>
<img src="/footer-badge.png" fetchpriority="low" loading="lazy" alt="Badge

<!— High priority: critical above-the-fold stylesheet —>
<link rel="preload" href="/critical.css" as="style" fetchpriority="high" /
```

The browser already prioritizes resources by type (CSS > JS > images), but `fetchpriority` lets you override within the same type. Your hero image should download before the 50 product thumbnails.

Compression — Brotli vs Gzip:

Brotli compresses 15-25% better than Gzip for text resources (HTML, CSS, JS, JSON). All modern browsers support it. Most CDNs and servers serve Brotli by default. Verify with `Content-Encoding: br` in response headers.

Reduce request count where it matters:

- Inline small SVG icons (< 1KB) instead of separate icon files
- Use CSS `background-image` for decorative images that don't need `alt` text
- Bundle related API calls into one request where the backend supports it
- Use `<link rel="modulepreload">` for ES module dependencies

Follow-up: "When would you use a CDN vs serving directly from your origin?"

Almost always use a CDN. CDNs cache static assets at edge locations geographically close to users, reducing latency from 200ms (cross-continent) to 20ms (nearest edge node). The origin server only handles the first request and cache misses. For dynamic content, CDNs can still help with connection reuse and TLS termination at the edge.

Q 8.10 How do you optimize React performance?

Difficulty: Mid **Asked at:** Every company

Preventing unnecessary re-renders:

React re-renders a component when its state changes or its parent re-renders. Most performance issues come from components re-rendering when nothing they display has actually changed.

JSX Copy

```jsx
// Problem: ExpensiveList re-renders every time parent state changes,
// even if `items` hasn't changed
function Parent() {
  const [count, setCount] = useState(0);
  const items = useItems(); // Assume this returns the same reference

  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>{count}</button>
      <ExpensiveList items={items} /> {/* Re-renders on every click! */}
    </>
  );
}
```

```
  return items.map(item => <ListItem key={item.id} item={item} />);
});
```

`useMemo` and `useCallback` — stabilize references:

JSX Copy

```jsx
function ProductFilter({ products, category }) {
  // Without useMemo: filtered is a new array on every render,
  // causing ExpensiveList to re-render even if products/category haven't
  const filtered = useMemo(
    () => products.filter(p => p.category === category),
    [products, category]
  );

  // Without useCallback: new function reference on every render
  const handleClick = useCallback(
    (id) => selectProduct(id),
    []
  );

  return <ExpensiveList items={filtered} onItemClick={handleClick} />;
}
```

Only use `useMemo` / `useCallback` when passing values to memoized children or when the computation is genuinely expensive. Wrapping everything in `useMemo` adds overhead without benefit if the child isn't memoized.

Virtualization for long lists:

Rendering 10,000 DOM elements is slow no matter how optimized your React is. Virtualization renders only the visible items (plus a small buffer) and swaps elements as the user scrolls.

JSX Copy

```
function VirtualList({ items }) {
  const parentRef = useRef(null);

  const virtualizer = useVirtualizer({
    count: items.length,  // Could be 100,000
    getScrollElement: () => parentRef.current,
    estimateSize: () => 50,  // Estimated row height
  });

  return (
    <div ref={parentRef} style={{ height: '500px', overflow: 'auto' }}>
      <div style={{ height: virtualizer.getTotalSize() }}>
        {virtualizer.getVirtualItems().map(virtualRow => (
          <div key={virtualRow.key} style={{
            position: 'absolute',
            top: virtualRow.start,
            height: virtualRow.size,
          }}>
            {items[virtualRow.index].name}
          </div>
        ))}
      </div>
    </div>
  );
}
// Only ~20 DOM elements exist at any time, regardless of list size
```

`useTransition` for non-urgent updates:

JSX Copy

```
function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  function handleSearch(value) {
    setQuery(value);  // Urgent: update the input immediately

    startTransition(() => {
```

```
  return (
    <>
      <input value={query} onChange={e => handleSearch(e.target.value)} />
      {isPending ? <Spinner /> : <ProductList results={results} />}
    </>
  );
}
```

The input stays responsive while the expensive filtering happens in a lower-priority update that React can interrupt if the user types again.

Follow-up: "How do you find which component is causing performance issues?"

React DevTools Profiler. Record a session, interact with the page, and the Profiler shows which components rendered, why they rendered (props changed, state changed, parent rendered), and how long each render took. Sort by render duration to find the bottlenecks.

Q 8.11 How do you identify and fix memory leaks in frontend applications?

Difficulty: Senior **Asked at:** Google, Amazon, Flipkart, Microsoft

Memory leaks happen when your application holds references to objects that are no longer needed, preventing the garbage collector from freeing that memory. Over time, memory usage grows, the page gets sluggish, and eventually the tab crashes.

Common causes in frontend applications:

1. Event listeners not cleaned up:

JAVASCRIPT Copy

```
  window.addEventListener('resize', handleResize);
  // Missing cleanup!
}, []);

// FIX:
useEffect(() => {
  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, []);
```

2. Closures holding references to detached DOM:

JAVASCRIPT Copy

```
function createHandler() {
  const largeElement = document.querySelector('.huge-list');

  return function handler() {
    // This closure holds a reference to largeElement
    // Even after largeElement is removed from DOM, it stays in memory
    console.log(largeElement.children.length);
  };
}
```

3. Forgotten timers and intervals:

JAVASCRIPT Copy

```
// LEAK: Interval keeps running after component unmounts
useEffect(() => {
  const id = setInterval(pollData, 5000);
  // Missing: return () => clearInterval(id);
}, []);
```

4. Growing data structures without bounds:

JAVASCRIPT Copy

```
const cache = new Map();
function fetchData(key) {
  if (cache.has(key)) return cache.get(key);
  const data = expensiveFetch(key);
  cache.set(key, data);  // Never evicted — Map grows forever
  return data;
}

// FIX: Use WeakMap (values are garbage collected when keys are GC'd)
// Or implement an LRU cache with a size limit
```

How to detect memory leaks:

Chrome DevTools → Memory tab → take a heap snapshot before an action, perform the action (e.g., open and close a modal 10 times), take another snapshot, and compare. Growing object counts or detached DOM nodes indicate leaks.

The Performance Monitor (DevTools → More tools → Performance monitor) shows real-time JS heap size. If it trends upward during normal usage without plateauing, you have a leak.

Follow-up: "What are `WeakRef` and `WeakMap` and how do they help?"

`WeakMap` and `WeakSet` hold "weak" references that don't prevent garbage collection. If the key of a WeakMap entry is no longer referenced anywhere else, both the key and value are garbage collected. This makes WeakMap ideal for caches and metadata attached to DOM elements. `WeakRef` lets you hold a reference to an object without preventing its garbage collection — useful for caches where you want to use the object if it's still alive but don't need to keep it alive.

Q 8.12 How do you handle third-party scripts without killing performance?

Difficulty: Mid **Asked at:** Google, Flipkart, Amazon, Swiggy

Third-party scripts (analytics, ads, chat widgets, A/B testing) are the biggest uncontrolled performance risk. A single poorly written third-party script can add 500ms to load time, block the main thread for 200ms, and cause layout shifts.

The problem quantified: The average website loads 20+ third-party scripts. They collectively add 1-3 seconds to page load and account for 50-70% of JavaScript execution time on many sites.

Strategies:

1. `async` and `defer` — minimum effort:

HTML Copy

```html
<!-- Analytics: independent, doesn't need DOM, order doesn't matter -->
<script async src="https://analytics.example.com/tracker.js"></script>

<!-- A/B testing: needs to run before paint to avoid flicker -->
<!-- This one genuinely needs to be synchronous - rare exception -->
<script src="https://ab-test.example.com/sdk.js"></script>
```

2. Facades — load on interaction, not on page load:

A chat widget doesn't need to load until the user wants to chat. Show a fake (facade) button that looks like the real widget. When clicked, load the real script.

JSX Copy

```jsx
function ChatWidget() {
  const [loaded, setLoaded] = useState(false);

  if (!loaded) {
    return (
      <button onClick={() => setLoaded(true)} className="chat-facade">
        💬 Chat with us
      </button>
    );
  }

  // Load the real widget only on interaction
```

This pattern saves 200-500KB of JavaScript and several seconds of main thread time for users who never click the chat button.

3. Partytown — run third-party scripts in a Web Worker:

Partytown moves third-party scripts off the main thread entirely. Analytics, tracking, and ad scripts run in a Web Worker, keeping the main thread free for your application code.

4. Performance budgets for third-party:

Set hard limits. "Total third-party JavaScript must be under 100KB." "No third-party script may block the main thread for more than 50ms." Enforce these in CI with Lighthouse CI or bundlesize.

Follow-up: "How do you convince product/marketing teams to limit third-party scripts?"

Show them the data. Run a performance test with and without the script. Show the impact on Core Web Vitals, conversion rate (Google reports a 7% conversion drop per second of load time), and SEO ranking. Frame it as business impact, not technical debt.

QUICK CHECK — no marks, just for you.

1. Your app needs to flush a queue of analytics events. Why does the lesson say requestIdleCallback is the right tool here, but the wrong tool for rendering search results the user just requested?

- ○ Idle callbacks run only when the browser has free time, so there is no guarantee about when they will fire — fine for background work, unacceptable for work a user is waiting on
- ○ requestIdleCallback runs on a separate thread, so it cannot touch anything that ends up in the DOM
- ○ Idle callbacks fire before paint, so anything rendered inside them is discarded on the next frame

requestIdleCallback is capped at 50ms of execution, so any work longer than that is silently dropped

Check answer

2. A long marketing page has 50 stacked sections. You add content-visibility: auto to each section. What does the browser actually do differently as a result?

- It removes offscreen sections from the DOM entirely and re-inserts them when they approach the viewport
- It skips layout, paint, and style calculation for sections that are not near the viewport, rendering them only as the user scrolls close to them
- It downloads each section's images only once that section scrolls into view, leaving layout work unchanged
- It moves each section's rendering onto its own GPU layer so scrolling is composited rather than repainted

Check answer

3. An API response is served with Cache-Control: public, max-age=300, stale-while-revalidate=600. A request for that resource arrives 500 seconds after it was cached. What happens?

- The cached (stale) response is served immediately while the browser fetches a fresh copy in the background
- The cached response is served and no revalidation happens until 900 seconds have passed
- The browser blocks and waits for a fresh response from the server, since max-age has expired
- The cache entry was evicted at 300 seconds, so this is a full cache miss and a normal 200 response is fetched

Previous
Rendering and loading

Next
Measurement and strategy

LEARN MORE

ARTICLES & DOCS

Web performance (MDN)
MDN

VIDEOS



Optimize network performance for web apps
YouTube



Front-end performance: techniques & tools
YouTube



10 frontend performance tips
YouTube

Hand-picked references for this lesson.